

Complemento: Regresión Binaria

Jorge Luis Bazán

08-06-2026

Índice

Carga de librerías

0. Ejemplo comparando lm con glm

1. Ejemplo 3.1: Dosis Insecticidas (Krzanowski 1998)

2. Ejemplo 3.2: Data vasoconstricción (Krzanowski 1998)

3.2.1 Tabla estimaciones y residuos

3. Ejemplo 4.1: Funcionamiento turbinas (Dunn & Smyth 2018)

Curva ROC

4. Ejercicio Propuesto

5. Bibliografía

Carga de librerías

Las librerías a utilizar son las siguientes

```
library(ISLR)
library(MASS)
library(pROC)
library(GLMsData)
```

0. Ejemplo comparando lm con glm

Recordemos los siguientes datos en la clase anterior

```
data("PimaIndiansDiabetes")
datos.pima<-PimaIndiansDiabetes
head(datos.pima)
```

	pregnant	glucose	pressure	triceps	insulin	mass	pedigree	age	diabetes
## 1	6	148	72	35	0	33.6	0.627	50	pos
## 2	1	85	66	29	0	26.6	0.351	31	neg
## 3	8	183	64	0	0	23.3	0.672	32	pos
## 4	1	89	66	23	94	28.1	0.167	21	neg
## 5	0	137	40	35	168	43.1	2.288	33	pos
## 6	5	116	74	0	0	25.6	0.201	30	neg

Aplicamos una regresión lineal usando lm

```
lm4<-lm(glucose~age+mass+triceps,data=datos.pima)
summary(lm4)
```

```
##
## Call:
## lm(formula = glucose ~ age + mass + triceps, data = datos.pima)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -132.441  -19.076   -1.995   18.303   83.776
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  70.290238   5.405596  13.003 < 2e-16 ***
## age          0.696954   0.093545   7.450 2.52e-13 ***
## mass         0.852423   0.150726   5.655 2.20e-08 ***
## triceps      0.008069   0.074933   0.108  0.914
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 30.15 on 764 degrees of freedom
## Multiple R-squared:  0.1143, Adjusted R-squared:  0.1108
## F-statistic: 32.85 on 3 and 764 DF, p-value: < 2.2e-16
```

Encontramos que el R^2 ajustado es 0.1108 lo que indica que el modelo no ajusta bien a los datos. La hipótesis fue si los coeficientes de regresión son iguales a cero. En este se rechaza H_0 . Hay efecto de regresión pero su ajuste es bajo.

Aplicamos una regresión lineal usando glm

```
glm4<-glm(glucose~age+mass+triceps,data=datos.pima)
summary(glm4)
```

```
##
## Call:
## glm(formula = glucose ~ age + mass + triceps, data = datos.pima)
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  70.290238   5.405596  13.003 < 2e-16 ***
## age          0.696954   0.093545   7.450 2.52e-13 ***
## mass         0.852423   0.150726   5.655 2.20e-08 ***
## triceps      0.008069   0.074933   0.108  0.914
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for gaussian family taken to be 909.0087)
##
##      Null deviance: 784064  on 767  degrees of freedom
## Residual deviance: 694483  on 764  degrees of freedom
## AIC: 7417.4
##
## Number of Fisher Scoring iterations: 2
```

```
anova(glm4, test = "Chisq")
```

```
## Analysis of Deviance Table
##
## Model: gaussian, link: identity
##
## Response: glucose
##
## Terms added sequentially (first to last)
##
##
##      Df Deviance Resid. Df Resid. Dev  Pr(>Chi)
## NULL                                767      784064
## age      1      54445      766      729619 1.000e-14 ***
## mass     1      35126      765      694493 5.091e-10 ***
## triceps  1         11      764      694483  0.9142
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Encontramos que el desvío residual es 694483 y AIC es 7417.4

1. Ejemplo 3.1: Dosis Insecticidas (Krzanowski 1998)

Respecto a este ejemplo cargamos primero los datos:

```
data.dosis <- data.frame(
  insecticida = rep(c("A", "B", "C"), each = 6),
  dosis = rep(c(2.00, 2.64, 3.48, 4.59, 6.06, 8.00), times = 3),
  muertos = c(
    3, 5, 19, 19, 24, 35,
    2, 14, 20, 27, 41, 40,
    28, 37, 46, 48, 48, 50
  ),
  total = c(
    50, 49, 47, 38, 49, 50,
    50, 49, 50, 50, 50, 50,
    50, 50, 50, 50, 50, 50
  )
)
data.dosis
```

```
##      insecticida dosis muertos total
## 1          A    2.00         3     50
## 2          A    2.64         5     49
## 3          A    3.48        19     47
## 4          A    4.59        19     38
## 5          A    6.06        24     49
## 6          A    8.00        35     50
## 7          B    2.00         2     50
## 8          B    2.64        14     49
## 9          B    3.48        20     50
## 10         B    4.59        27     50
## 11         B    6.06        41     50
## 12         B    8.00        40     50
## 13         C    2.00        28     50
## 14         C    2.64        37     50
## 15         C    3.48        46     50
## 16         C    4.59        48     50
## 17         C    6.06        48     50
## 18         C    8.00        50     50
```

```
head(data.dosis)
```

```
##      insecticida dosis muertos total
## 1          A    2.00         3     50
## 2          A    2.64         5     49
## 3          A    3.48        19     47
## 4          A    4.59        19     38
## 5          A    6.06        24     49
## 6          A    8.00        35     50
```

Agregamos log dosis como covariable:

```
data.dosis$logdosis<-log(data.dosis$dosis)
```

Ajustamos el modelo general que contemple intercepto y pendiente por cada insecticida, considerando y_{ij} : insectos muertos de muestra i de insecticida j ,

$$\begin{aligned}
 M_1 : \quad y_{ij} &\sim \text{Binomial}(n_{ij}, \pi_{ij}) \\
 g(\pi_{ij}) &= \text{logit}(\pi_{ij}) = \eta_{ij} \\
 \eta_{ij} &= \mathbf{x}_{ij}^\top \boldsymbol{\beta}_j \\
 i &= 1, \dots, 6; \quad j = 1, 2, 3
 \end{aligned}$$

Ajustamos el modelo general y exploramos sus resultados:

```
# Ajuste modelo global
modelo.sep <- glm(
  cbind(muertos, total - muertos) ~
    insecticida + insecticida:logdosis,
  family = binomial(link = "logit"),
```

```

data = data.dosis
)

summary(modelo.sep)

##
## Call:
## glm(formula = cbind(muertos, total - muertos) ~ insecticida +
##      insecticida:logdosis, family = binomial(link = "logit"),
##      data = data.dosis)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -3.8308     0.5002  -7.659 1.87e-14 ***
## insecticidaB    -0.2120     0.7053  -0.301  0.7637
## insecticidaC     1.7101     0.7741   2.209  0.0272 *
## insecticidaA:logdosis  2.2824     0.3190   7.156 8.33e-13 ***
## insecticidaB:logdosis  2.8381     0.3392   8.367 < 2e-16 ***
## insecticidaC:logdosis  3.3884     0.5733   5.910 3.42e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 407.129  on 17  degrees of freedom
## Residual deviance:  19.292  on 12  degrees of freedom
## AIC: 94.551
##
## Number of Fisher Scoring iterations: 4

```

```
AIC(modelo.sep)
```

```
## [1] 94.55061
```

```
BIC(modelo.sep)
```

```
## [1] 99.89284
```

La devianza residual obtenida es de 19.29 con 12 grados de libertad, el AIC es de 94.55, BIC=99.892

Imponemos ahora la condición de *líneas paralelas*, esto es asumir que el log de la dosis afecta de la misma manera en todos los insecticidas: $\beta_{11} = \beta_{12} = \beta_{13}$

```

# Modelo líneas paralelas
modelo.par <- glm(
  cbind(muertos, total - muertos) ~
    insecticida + logdosis,
  family = binomial(link = "logit"),
  data = data.dosis
)
summary(modelo.par)

```

```
##
## Call:
## glm(formula = cbind(muertos, total - muertos) ~ insecticida +
##      logdosis, family = binomial(link = "logit"), data = data.dosis)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -4.4541      0.3575 -12.460 < 2e-16 ***
## insecticidaB   0.6144      0.1999   3.074 0.00211 **
## insecticidaC   3.0314      0.2521  12.022 < 2e-16 ***
## logdosis       2.6938      0.2146  12.551 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 407.129  on 17  degrees of freedom
## Residual deviance:  22.685  on 14  degrees of freedom
## AIC: 93.943
##
## Number of Fisher Scoring iterations: 4
```

```
AIC(modelo.par)
```

```
## [1] 93.9431
```

```
BIC(modelo.par)
```

```
## [1] 97.50459
```

Recordemos que $AIC = -2ll + 2p$, donde p es el número de parametros. Otra alternativa es el $BIC = -2ll + p \log(n)$. En ambos casos cuanto menor es AIC o BIC, mejor es el modelo.

Vemos ahora que la devianza ahora es de 22.68 con 14 grados de libertad, y el AIC es de 93.94 y BIC=97.5. Comparando con el modelo anterior vemos que presenta peor ajuste tanto en términos de AIC como también debido al aumento de la devianza, sin embargo respecto a esto último podemos probar la significancia de este aumento, considerando que asintóticamente

$$\Delta D = D_{M_2} - D_{M_1} \sim \chi_{df_2 - df_1}^2$$

Dado la naturaleza de M_2 y M_1 en este caso probar si $\Delta D = 0$ es equivalente a probar

$$H_0 : \beta_{11} = \beta_{12} = \beta_{13} \quad \text{v/s} \quad H_1 : \beta_{1i} \neq \beta_{1j} \quad \text{para algún } i, j$$

Lo probamos con el comando `anova()`:

```
anova(modelo.par, modelo.sep, test = "Chisq")
```

```
## Analysis of Deviance Table
```

```
##
```

```
## Model 1: cbind(muertos, total - muertos) ~ insecticida + logdosis
```

```
## Model 2: cbind(muertos, total - muertos) ~ insecticida + insecticida:logdosis
##   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
## 1      14      22.685
## 2      12      19.292  2   3.3925   0.1834
```

Basado en el valor-p, aceptamos que los desvios de los modelos comparados son iguales. Entonces, vemos que con un 5 % de significancia la diferencia entre los modelos no resulta significativa por lo que no podemos rechazar la hipótesis que oos insecticidas afecten de manera similar cada uno de ellos. Aunque hay algunas pequeñas diferencias en AIC y BIC, la prueba de hipótesis muestra que los modelos son similares. En estos casos se prefiere modelos parsimoniosos (mas simples).

```
modelo.comun <- glm(
  cbind(muertos, total - muertos) ~ logdosis,
  family = binomial(link = "logit"),
  data = data.dosis
)
summary(modelo.comun)
```

```
##
## Call:
## glm(formula = cbind(muertos, total - muertos) ~ logdosis, family = binomial(link = "logit"),
##      data = data.dosis)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -2.4817     0.2396  -10.36  <2e-16 ***
## logdosis       2.0559     0.1723   11.93  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 407.13  on 17  degrees of freedom
## Residual deviance: 235.28  on 16  degrees of freedom
## AIC: 302.53
##
## Number of Fisher Scoring iterations: 5
```

```
AIC(modelo.comun)
```

```
## [1] 302.5334
```

```
BIC(modelo.comun)
```

```
## [1] 304.3142
```

Podemos comparar ahora los 3 modelos:

```
anova(modelo.comun, modelo.par, modelo.sep,
  test = "Chisq")
```

```
## Analysis of Deviance Table
##
## Model 1: cbind(muertos, total - muertos) ~ logdosis
## Model 2: cbind(muertos, total - muertos) ~ insecticida + logdosis
## Model 3: cbind(muertos, total - muertos) ~ insecticida + insecticida:logdosis
##   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
## 1         16      235.275
## 2         14       22.685  2   212.590  <2e-16 ***
## 3         12       19.292  2     3.392  0.1834
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Comparando ahora los 3 modelos, donde el último corresponde simplemente a una línea general para todos los insecticidas, se ve que hay diferencias significativas entre este y el que incorpora al insecticida como predictor, sin embargo entre este modelo y el que propone distintas relaciones entre la log dosis y la proporción de insectos según insecticida no hay diferencias significativas.

2. Ejemplo 3.2: Data vasoconstricción (Krzanowski 1998)

```
data.vaso <- data.frame(
  i = 1:39,
  x1 = c(
    3.70, 3.50, 1.25, 0.75, 0.80,
    0.70, 0.60, 1.10, 0.90, 0.90,
    0.80, 0.55, 0.60, 1.40, 0.75,
    2.30, 3.20, 0.85, 1.70, 1.80,
    0.40, 0.95, 1.35, 1.50, 1.60,
    0.60, 1.80, 0.95, 1.90, 1.60,
    2.70, 2.35, 1.10, 1.10, 1.20,
    0.80, 0.95, 0.75, 1.30
  ),
  x2 = c(
    0.83, 1.09, 2.50, 1.50, 3.20,
    3.50, 0.75, 1.70, 0.75, 0.45,
    0.57, 2.75, 3.00, 2.33, 3.75,
    1.64, 1.60, 1.42, 1.06, 1.80,
    2.00, 1.36, 1.35, 1.36, 1.78,
    1.50, 1.50, 1.90, 0.95, 0.40,
    0.75, 0.03, 1.83, 2.20, 2.00,
    3.33, 1.90, 1.90, 1.63
  ),
  y = c(
    1, 1, 1, 1, 1,
    1, 0, 0, 0, 0,
    0, 0, 0, 1, 1,
    1, 1, 1, 0, 1,
    0, 0, 0, 0, 1,
    0, 1, 0, 1, 0,
    1, 0, 0, 1, 1,
    1, 0, 0, 1
  ),
)
```



```

pi_hat = c(
  0.999, 0.999, 0.875, 0.066, 0.886,
  0.921, 0.005, 0.320, 0.017, 0.008,
  0.007, 0.473, 0.679, 0.889, 0.965,
  0.977, 0.999, 0.077, 0.470, 0.903,
  0.064, 0.096, 0.329, 0.474, 0.802,
  0.038, 0.807, 0.308, 0.590, 0.095,
  0.950, 0.419, 0.399, 0.638, 0.605,
  0.917, 0.308, 0.170, 0.456
),
r_pearson = c(
  0.030, 0.031, 0.378, 3.751, 0.358,
  0.292, -0.074, -0.685, -0.132, -0.089,
  -0.086, -0.947, -1.453, 0.354, 0.191,
  0.154, 0.028, 3.458, -0.941, 0.328,
  -0.262, -0.326, -0.700, -0.950, 0.497,
  -0.199, 0.489, -0.668, 0.834, -0.323,
  0.230, -0.849, -0.814, 0.752, 0.808,
  0.302, -0.668, -0.453, 1.093
)
)
data.vaso

```

```

##      i   x1   x2 y pi_hat r_pearson
## 1    1  3.70 0.83 1  0.999      0.030
## 2    2  3.50 1.09 1  0.999      0.031
## 3    3  1.25 2.50 1  0.875      0.378
## 4    4  0.75 1.50 1  0.066      3.751
## 5    5  0.80 3.20 1  0.886      0.358
## 6    6  0.70 3.50 1  0.921      0.292
## 7    7  0.60 0.75 0  0.005     -0.074
## 8    8  1.10 1.70 0  0.320     -0.685
## 9    9  0.90 0.75 0  0.017     -0.132
## 10   10 0.90 0.45 0  0.008     -0.089
## 11   11 0.80 0.57 0  0.007     -0.086
## 12   12 0.55 2.75 0  0.473     -0.947
## 13   13 0.60 3.00 0  0.679     -1.453
## 14   14 1.40 2.33 1  0.889      0.354
## 15   15 0.75 3.75 1  0.965      0.191
## 16   16 2.30 1.64 1  0.977      0.154
## 17   17 3.20 1.60 1  0.999      0.028
## 18   18 0.85 1.42 1  0.077      3.458
## 19   19 1.70 1.06 0  0.470     -0.941
## 20   20 1.80 1.80 1  0.903      0.328
## 21   21 0.40 2.00 0  0.064     -0.262
## 22   22 0.95 1.36 0  0.096     -0.326
## 23   23 1.35 1.35 0  0.329     -0.700
## 24   24 1.50 1.36 0  0.474     -0.950
## 25   25 1.60 1.78 1  0.802      0.497
## 26   26 0.60 1.50 0  0.038     -0.199
## 27   27 1.80 1.50 1  0.807      0.489
## 28   28 0.95 1.90 0  0.308     -0.668
## 29   29 1.90 0.95 1  0.590      0.834

```

```
## 30 30 1.60 0.40 0 0.095 -0.323
## 31 31 2.70 0.75 1 0.950 0.230
## 32 32 2.35 0.03 0 0.419 -0.849
## 33 33 1.10 1.83 0 0.399 -0.814
## 34 34 1.10 2.20 1 0.638 0.752
## 35 35 1.20 2.00 1 0.605 0.808
## 36 36 0.80 3.33 1 0.917 0.302
## 37 37 0.95 1.90 0 0.308 -0.668
## 38 38 0.75 1.90 0 0.170 -0.453
## 39 39 1.30 1.63 1 0.456 1.093
```

```
head(data.vaso)
```

```
##   i   x1   x2 y pi_hat r_pearson
## 1 1 3.70 0.83 1 0.999 0.030
## 2 2 3.50 1.09 1 0.999 0.031
## 3 3 1.25 2.50 1 0.875 0.378
## 4 4 0.75 1.50 1 0.066 3.751
## 5 5 0.80 3.20 1 0.886 0.358
## 6 6 0.70 3.50 1 0.921 0.292
```

Ajustamos el modelo general basado en una regresión binaria con enlace logístico y exploramos sus resultados:

```
# Ajuste modelo global
modelo.vaso <- glm(
  y ~ x1 + x2,
  family = binomial(link = "logit"),
  data = data.vaso
)
summary(modelo.vaso)
```

```
##
## Call:
## glm(formula = y ~ x1 + x2, family = binomial(link = "logit"),
##      data = data.vaso)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -9.5536      3.2419  -2.947  0.00321 **
## x1              3.8907      1.4319   2.717  0.00658 **
## x2              2.6561      0.9166   2.898  0.00376 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 54.040  on 38  degrees of freedom
## Residual deviance: 29.733  on 36  degrees of freedom
## AIC: 35.733
##
## Number of Fisher Scoring iterations: 6
```

3.2.1 Tabla estimaciones y residuos

Replicando la tabla 3.1 del material obtenemos las probabilidades predichas por el modelo, los residuos y ordenamos en una tabla:

```
# Probabilidades predichas:
data.vaso$pi_hat <- fitted(modelo.vaso)

# Residuos
data.vaso$r_pearson <- rstandard(
  modelo.vaso,
  type = "pearson"
)

tabla.vaso <- data.frame(
  i = data.vaso$i,
  x1 = round(data.vaso$x1, 2),
  x2 = round(data.vaso$x2, 2),
  y = data.vaso$y,
  pi_hat = round(data.vaso$pi_hat, 3),
  r_pearson = round(data.vaso$r_pearson, 3)
)
```

Observando las primeras 20 filas:

```
tabla.vaso[1:20, ]
```

##	i	x1	x2	y	pi_hat	r_pearson
## 1	1	3.70	0.83	1	0.999	0.030
## 2	2	3.50	1.09	1	0.999	0.031
## 3	3	1.25	2.50	1	0.875	0.391
## 4	4	0.75	1.50	1	0.066	3.894
## 5	5	0.80	3.20	1	0.887	0.378
## 6	6	0.70	3.50	1	0.922	0.308
## 7	7	0.60	0.75	0	0.005	-0.074
## 8	8	1.10	1.70	0	0.319	-0.706
## 9	9	0.90	0.75	0	0.017	-0.134
## 10	10	0.90	0.45	0	0.008	-0.089
## 11	11	0.80	0.57	0	0.007	-0.086
## 12	12	0.55	2.75	0	0.473	-1.034
## 13	13	0.60	3.00	0	0.679	-1.589
## 14	14	1.40	2.33	1	0.889	0.366
## 15	15	0.75	3.75	1	0.965	0.196
## 16	16	2.30	1.64	1	0.977	0.157
## 17	17	3.20	1.60	1	0.999	0.028
## 18	18	0.85	1.42	1	0.078	3.567
## 19	19	1.70	1.06	0	0.469	-0.991
## 20	20	1.80	1.80	1	0.903	0.341

3. Ejemplo 4.1: Funcionamiento turbinas (Dunn & Smyth 2018)

Analizando la data de las turbinas, primero cargamos la data:

```
# Ejemplo 4.1 turbinas
data.turbina <- data.frame(
  caso = 1:11,
  hrs = c(
    400, 1000, 1400, 1800, 2200,
    2600, 3000, 3400, 3800, 4200, 4600
  ),
  ruedas = c(
    39, 53, 33, 73, 30,
    39, 42, 13, 34, 40, 36
  ),
  prop_fisuras = c(
    0.0000, 0.0755, 0.0606, 0.0959, 0.1667,
    0.2308, 0.2143, 0.4615, 0.6471, 0.5250, 0.5833
  ),
  n_fisuras = c(
    0, 4, 2, 7, 5,
    9, 9, 6, 22, 21, 21
  )
)
data.turbina
```

```
##      caso  hrs  ruedas prop_fisuras n_fisuras
## 1      1   400     39      0.0000         0
## 2      2  1000     53      0.0755         4
## 3      3  1400     33      0.0606         2
## 4      4  1800     73      0.0959         7
## 5      5  2200     30      0.1667         5
## 6      6  2600     39      0.2308         9
## 7      7  3000     42      0.2143         9
## 8      8  3400     13      0.4615         6
## 9      9  3800     34      0.6471        22
## 10     10  4200     40      0.5250        21
## 11     11  4600     36      0.5833        21
```

```
head(data.turbina)
```

```
##      caso  hrs  ruedas prop_fisuras n_fisuras
## 1      1   400     39      0.0000         0
## 2      2  1000     53      0.0755         4
## 3      3  1400     33      0.0606         2
## 4      4  1800     73      0.0959         7
## 5      5  2200     30      0.1667         5
## 6      6  2600     39      0.2308         9
```

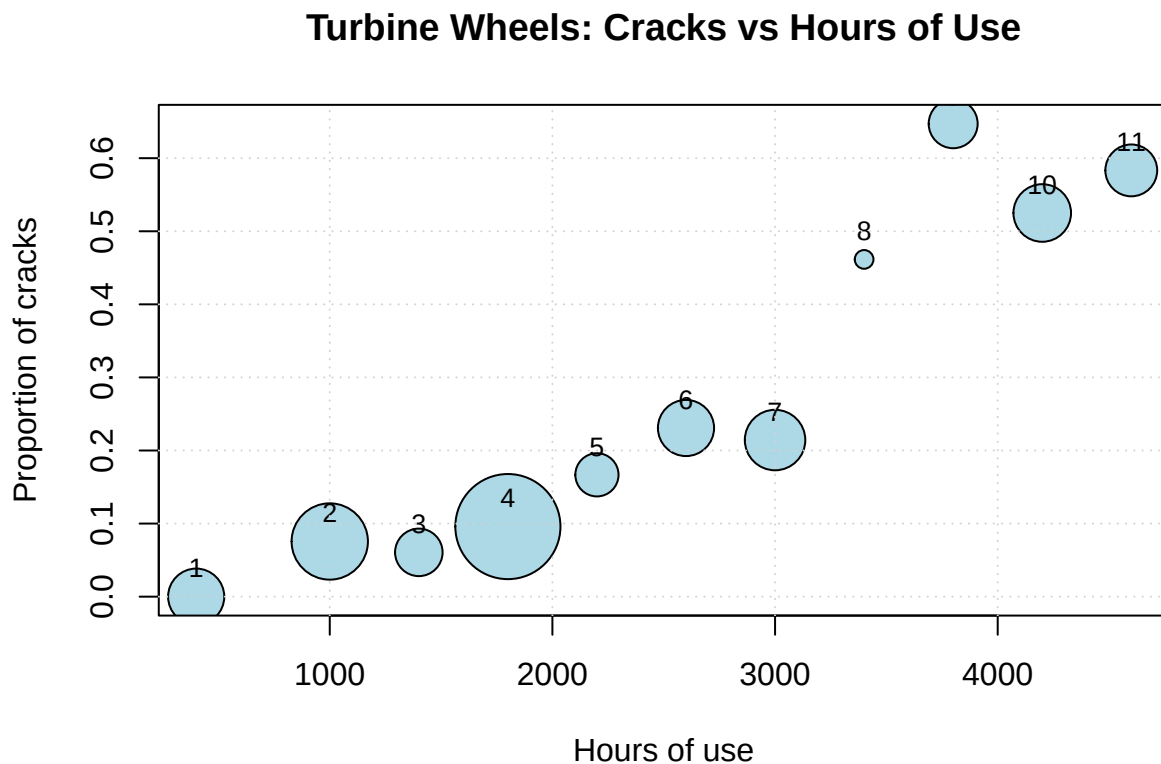
Viendo el Scatterplot entre las variables se puede pensar en enlaces sigmoidales:

```
# Gráfico Scatterplot
tam <- data.turbina$ruedas / 10
plot(
  data.turbina$hrs,
  data.turbina$prop_fisuras,
```

```

cex = tam,          # controls circle size
pch = 21,
bg = "lightblue",
xlab = "Hours of use",
ylab = "Proportion of cracks",
main = "Turbine Wheels: Cracks vs Hours of Use"
)
text(
  data.turbina$hrs,
  data.turbina$prop_fisuras,
  labels = data.turbina$caso,
  pos = 3,
  cex = 0.8
)
grid()

```



Pasan-
do ajustar los modelos GLM, usaremos una regresión binomial, exploramos las 3 funciones de enlace: logit, probit y cloglog. Ajustamos, almacenamos en objetos diferentes y exploramos sus resultados:

```

# Ajustando modelos
# Logit:
mod.logit <- glm(
  cbind(n_fisuras, ruedas - n_fisuras) ~ hrs,
  family = binomial(link = "logit"),
  data = data.turbina
)
summary(mod.logit)

```

```
##
```

```
## Call:
## glm(formula = cbind(n_fisuras, ruedas - n_fisuras) ~ hrs, family = binomial(link = "logit"),
##      data = data.turbina)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -3.9235966  0.3779589 -10.381  <2e-16 ***
## hrs          0.0009992  0.0001142   8.754  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 112.670  on 10  degrees of freedom
## Residual deviance:  10.331  on  9  degrees of freedom
## AIC: 49.808
##
## Number of Fisher Scoring iterations: 4
```

```
# Probit:
mod.probit <- glm(
  cbind(n_fisuras, ruedas - n_fisuras) ~ hrs,
  family = binomial(link = "probit"),
  data = data.turbina
)
summary(mod.probit)
```

```
##
## Call:
## glm(formula = cbind(n_fisuras, ruedas - n_fisuras) ~ hrs, family = binomial(link = "probit"),
##      data = data.turbina)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -2.2758075  0.1974187 -11.528  <2e-16 ***
## hrs          0.0005783  0.0000626   9.239  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 112.6700  on 10  degrees of freedom
## Residual deviance:   9.8148  on  9  degrees of freedom
## AIC: 49.292
##
## Number of Fisher Scoring iterations: 4
```

```
# Clog-log:
mod.cloglog <- glm(
  cbind(n_fisuras, ruedas - n_fisuras) ~ hrs,
  family = binomial(link = "cloglog"),
  data = data.turbina
)
summary(mod.cloglog)
```

```
##
## Call:
## glm(formula = cbind(n_fisuras, ruedas - n_fisuras) ~ hrs, family = binomial(link = "cloglog"),
##      data = data.turbina)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -3.603e+00  3.247e-01 -11.096  <2e-16 ***
## hrs          8.105e-04  9.013e-05   8.992  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 112.670  on 10  degrees of freedom
## Residual deviance:  12.228  on  9  degrees of freedom
## AIC: 51.705
##
## Number of Fisher Scoring iterations: 5
```

De los 3 modelos vemos que para las primeras 2 funciones de enlace en términos de AIC los modelos son similares, siendo muy levemente menor el que abarca enlace logit. Obtenemos ahora la tabla resumen que compare los estimadores de los coeficientes:

```
# Tabla resumen:
tabla.glm <- data.frame(
  Modelo = c("Logit", "Probit", "Clog-log"),

  Intercepto = c(
    coef(mod.logit)[1],
    coef(mod.probit)[1],
    coef(mod.cloglog)[1]
  ),

  Beta = c(
    coef(mod.logit)[2],
    coef(mod.probit)[2],
    coef(mod.cloglog)[2]
  ),

  Deviance = c(
    deviance(mod.logit),
    deviance(mod.probit),
    deviance(mod.cloglog)
  )
)

# Aproximando valores:
tabla.glm[,2:4] <- round(tabla.glm[,2:4], 4)

# Tabla final:
tabla.glm
```

```
##      Modelo Intercepto  Beta Deviance
```

```
## 1    Logit    -3.9236 1e-03  10.3315
## 2    Probit   -2.2758 6e-04   9.8148
## 3 Clog-log   -3.6033 8e-04  12.2279
```

```
AIC(mod.logit,mod.probit,mod.cloglog)
```

```
##           df      AIC
## mod.logit    2 49.80819
## mod.probit    2 49.29156
## mod.cloglog   2 51.70463
```

```
BIC(mod.logit,mod.probit,mod.cloglog)
```

```
##           df      BIC
## mod.logit    2 50.60398
## mod.probit    2 50.08735
## mod.cloglog   2 52.50042
```

De acuerdo a AIC y BIC el mejor modelo es la regresión con enlace probito.

Replicamos ahora el gráfico de curvas de los valores predichos por los distintos modelos:

```
# Comparando curvas ajustadas
hrs.grid <- seq(
  min(data.turbina$hrs),
  max(data.turbina$hrs),
  length = 500
)

# Predicted probabilities
p.logit <- predict(
  mod.logit,
  newdata = data.frame(hrs = hrs.grid),
  type = "response"
)

p.probit <- predict(
  mod.probit,
  newdata = data.frame(hrs = hrs.grid),
  type = "response"
)

p.cloglog <- predict(
  mod.cloglog,
  newdata = data.frame(hrs = hrs.grid),
  type = "response"
)

# Scatterplot
plot(
  data.turbina$hrs,
  data.turbina$prop_fisuras,
  pch = 19,
```



```

xlab = "Hours of use",
ylab = "Proportion of cracks",
ylim = c(0, 0.8)
)

# Case labels
text(
  data.turbina$hrs,
  data.turbina$prop_fisuras,
  labels = data.turbina$caso,
  pos = 3,
  cex = 0.8
)

# Fitted curves
lines(
  hrs.grid,
  p.logit,
  lwd = 2,
  col = "black",
  lty = 1
)

lines(
  hrs.grid,
  p.probit,
  lwd = 2,
  col = "darkred",
  lty = 2
)

lines(
  hrs.grid,
  p.cloglog,
  lwd = 2,
  col = "darkblue",
  lty = 3
)

# Legend
legend(
  "topleft",
  legend = c(
    "Logit",
    "Probit",
    "Complementary log-log"
  ),
  col = c(
    "black",
    "darkred",
    "darkblue"
  ),
  lty = 1:3,

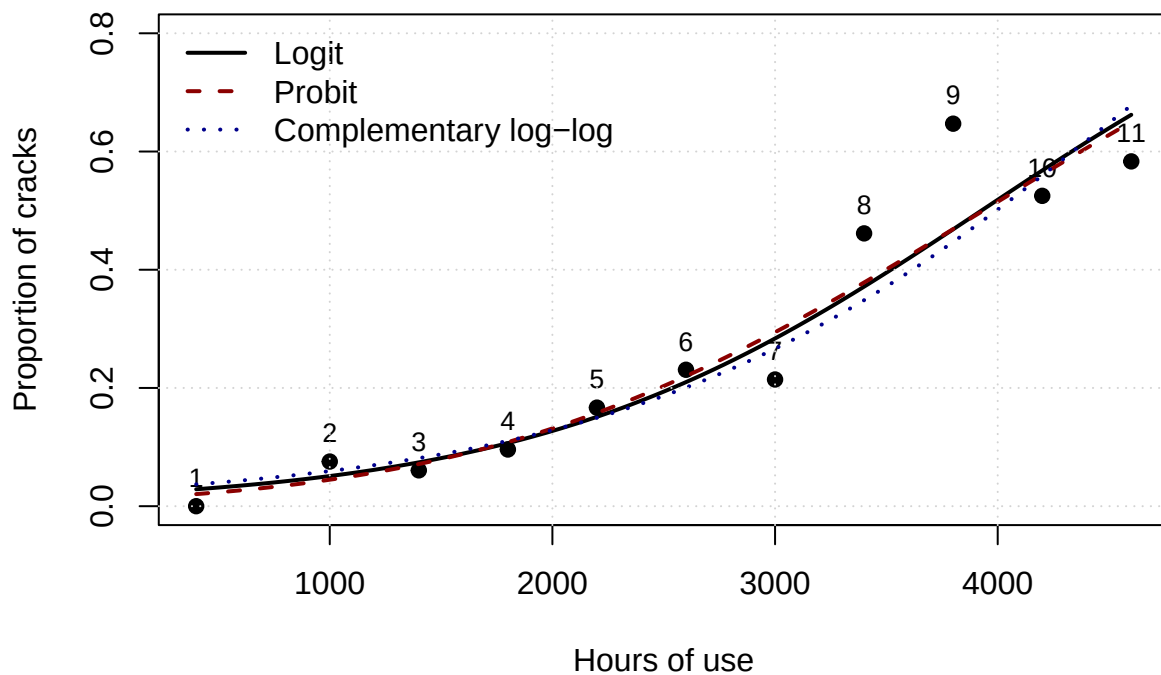
```

```

lwd = 2,
bty = "n"
)

grid()

```



Vemos que las 3 curvas presentan un comportamiento bastante similar, lo cual se condice con los valores similares de AIC observados anteriormente.

Curva ROC

Podemos también obtener la curva ROC para el enlace logit:

```

y.ind <- c()
hrs.ind <- c()

for(i in 1:nrow(data.turbina)) {

  # successes
  y.ind <- c(
    y.ind,
    rep(1, data.turbina$n_fisuras[i])
  )

  hrs.ind <- c(
    hrs.ind,
    rep(
      data.turbina$hrs[i],
      data.turbina$n_fisuras[i]
    )
  )
}

```

```

# failures
y.ind <- c(
  y.ind,
  rep(
    0,
    data.turbina$ruedas[i] -
    data.turbina$n_fisuras[i]
  )
)

hrs.ind <- c(
  hrs.ind,
  rep(
    data.turbina$hrs[i],
    data.turbina$ruedas[i] -
    data.turbina$n_fisuras[i]
  )
)
}

# Individual dataframe
data.ind <- data.frame(
  y = y.ind,
  hrs = hrs.ind
)

# Fit logistic model
mod.ind <- glm(
  y ~ hrs,
  family = binomial(link = "logit"),
  data = data.ind
)

# Predicted probabilities
p.hat <- predict(
  mod.ind,
  type = "response"
)

library(pROC)

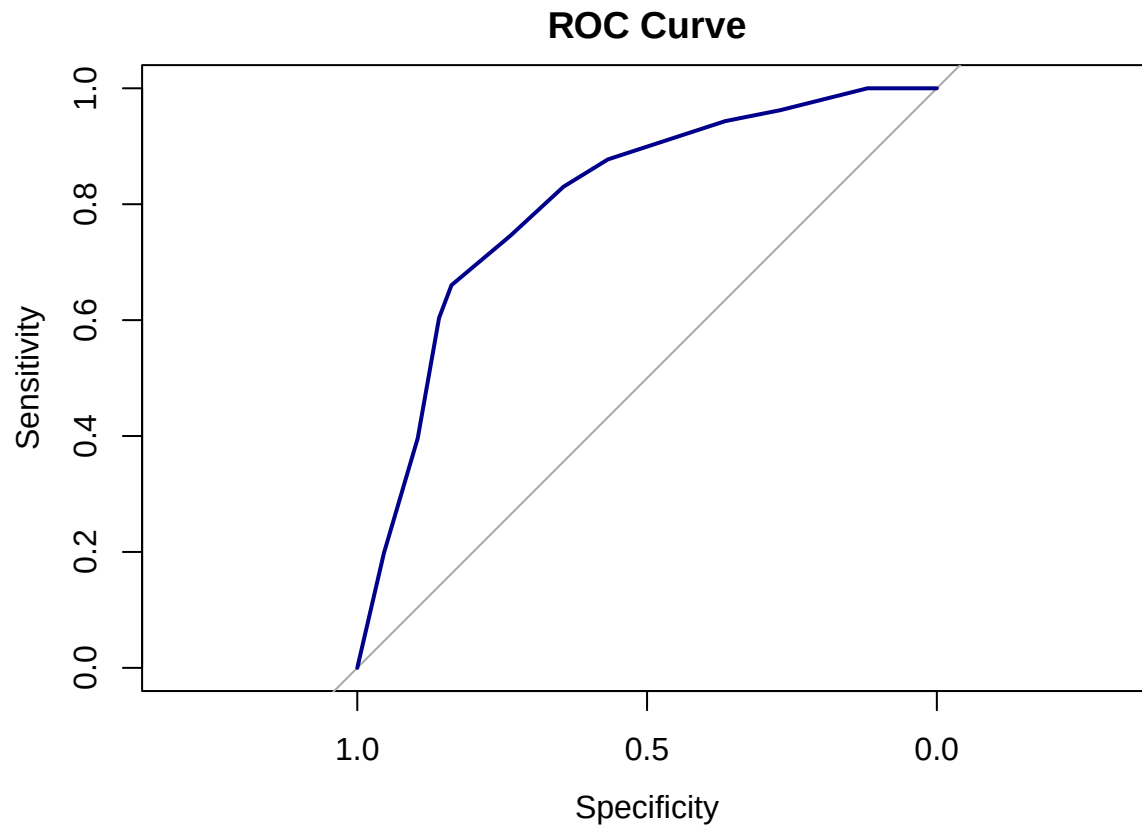
# ROC curve
roc.mod <- roc(
  response = data.ind$y,
  predictor = p.hat
)

```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
plot(
  roc.mod,
  col = "darkblue",
  lwd = 2,
  main = "ROC Curve"
)
```



Obteniendo el punto óptimo de threshold y con él los mejores valores de especificidad y sensibilidad:

```
best <- coords(
  roc.mod,
  x = "best",
  best.method = "youden",
  ret = c(
    "threshold",
    "sensitivity",
    "specificity",
    "accuracy"
  )
)
best
```

```
##           threshold sensitivity specificity accuracy
## threshold 0.3275831   0.6603774   0.8374233 0.7939815
```

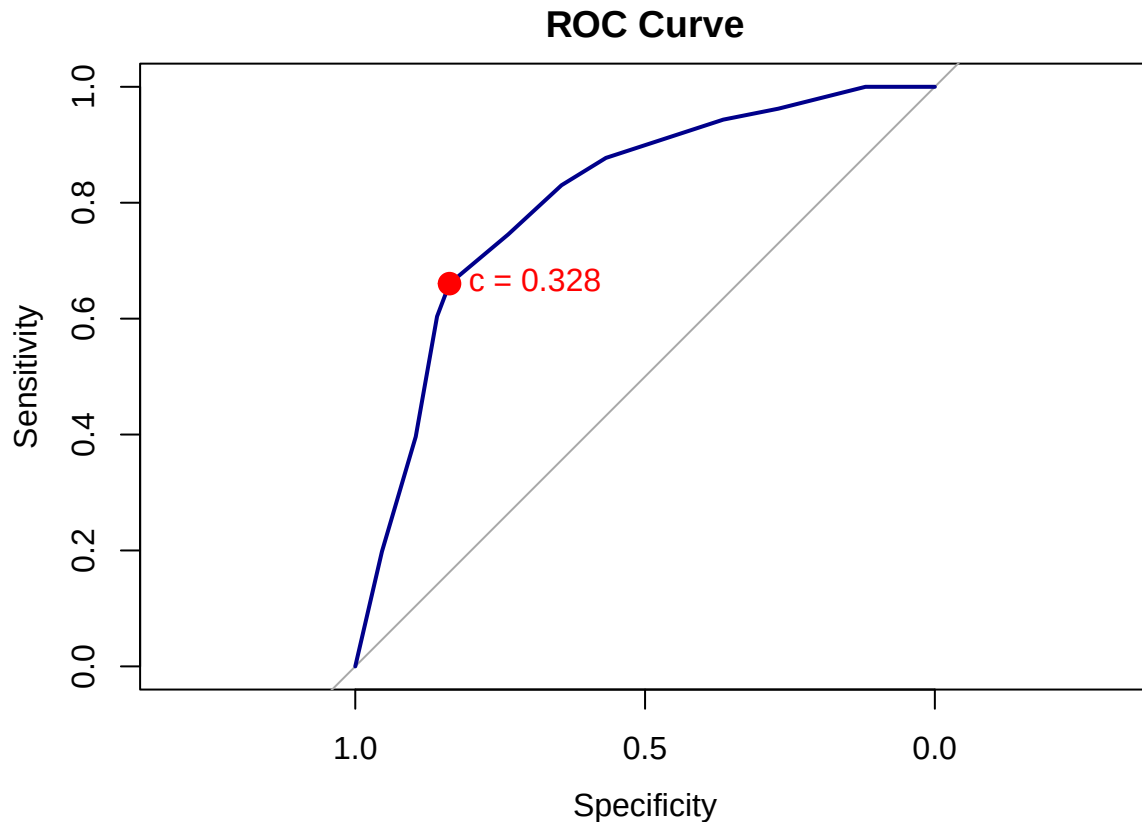
```
# Extract optimal values
threshold.opt <- best["threshold"]
sens.opt <- best["sensitivity"]
spec.opt <- best["specificity"]
acc.opt <- best["accuracy"]
```

Vemos que el valor umbral óptimo de probabilidad para realizar las clasificaciones es de 0.328 con lo cual se obtiene una accuracy de 79.4 %, sensibilidad de 0.66 y especificidad de 0.837. En la curva ROC esto es:

```
# Redraw ROC curve
plot(
  roc.mod,
  col = "darkblue",
  lwd = 2,
  main = "ROC Curve"
)

# Add optimal point
points(
  spec.opt,
  sens.opt,
  pch = 19,
  col = "red",
  cex = 1.5
)

# Add label
text(
  spec.opt,
  sens.opt,
  labels = paste0(
    "c = ",
    round(threshold.opt, 3)
  ),
  pos = 4,
  col = "red"
)
```



4. Ejercicio Propuesto

Considere los datos `birthwt` disponibles en el paquete `MASS` package documentation. Este conjunto de datos corresponde a un estudio sobre bajo peso al nacer en recién nacidos y contiene información materna y del embarazo.

La variable respuesta es `low` indicador binario que toma valor 1 si el peso del recién nacido es menor a 2500 gramos y 0 en caso contrario.

Se desea estudiar la relación entre la probabilidad de bajo peso al nacer y las características maternas, para esto considere:

1. Ajuste un modelo de regresión logística considerando como predictores `age`, `lwt`, `smoke`, `ht` y `ui`.
2. Escriba el modelo a emplear y ajústelo a los datos
3. Interprete los coeficientes asociados a `smoke` y `ht` en términos de odds ratios.
4. Evalúe la significancia de los parámetros
5. Calcule las probabilidades ajustadas para las observaciones y construya la curva ROC correspondiente.
6. Determine el threshold óptimo utilizando el índice de Youden e informe sensibilidad, especificidad y accuracy.
7. Analice la presencia de posibles observaciones influyentes utilizando residuos de Pearson o desvío.

5. Bibliografia

- Dunn, P., & Smyth, G. (2018). *Generalized Linear Models With Examples in R*. Springer-Verlag.
- James, G., Witten, D., Hastie, T., and Tibshirani, R. (2013) *An Introduction to Statistical Learning with applications in R*, <https://www.statlearning.com>. Springer-Verlag, New York.
- Krzanowski, W. (1998). *An Introduction to Statistical Modelling*. Arnold, New York.
- Siebert P (1987). “Vehicle Recognition Using Rule Based Methods”. Turing Institute Research Memorandum TIRM-87-018
- Blake CL, Merz CJ (1998). “UCI Repository of Machine Learning Databases”. University of California, Irvine, Department of Information and Computer Science. Formerly available from ‘<http://www.ics.uci.edu/~mlearn/MLRepository.html>’.